

Comparación de Estructuras de Caché: Boost Flat Map vs Google B-Tree

Evaluación técnica de rendimiento, latencia y tasa de aciertos en sistemas de almacenamiento temporal

Samuel Cordero y Dervis Martinez

Objetivos del Estudio Comparativo



Implementación Dual

Desarrollar dos sistemas de caché basados en FlatMap (estructura tipo Boost) y B-Tree (estructura tipo Google) para análisis comparativo directo



Medición de Tiempos

Evaluar precisamente los tiempos de inserción, búsqueda y eliminación en ambas estructuras bajo condiciones controladas



Métricas de Rendimiento

Calcular latencia promedio por operación y tasa de aciertos (hit rate) para determinar eficiencia operacional



Análisis Comparativo

Generar comparaciones exhaustivas de desempeño para identificar escenarios óptimos de uso para cada estructura



Descripción de las Estructuras de Datos

Cada estructura presenta características arquitectónicas distintivas que definen su comportamiento y casos de uso óptimos:

Flat Map (Boost)



Arquitectura: Arreglo ordenado con búsqueda binaria que mantiene los elementos contiguos en memoria

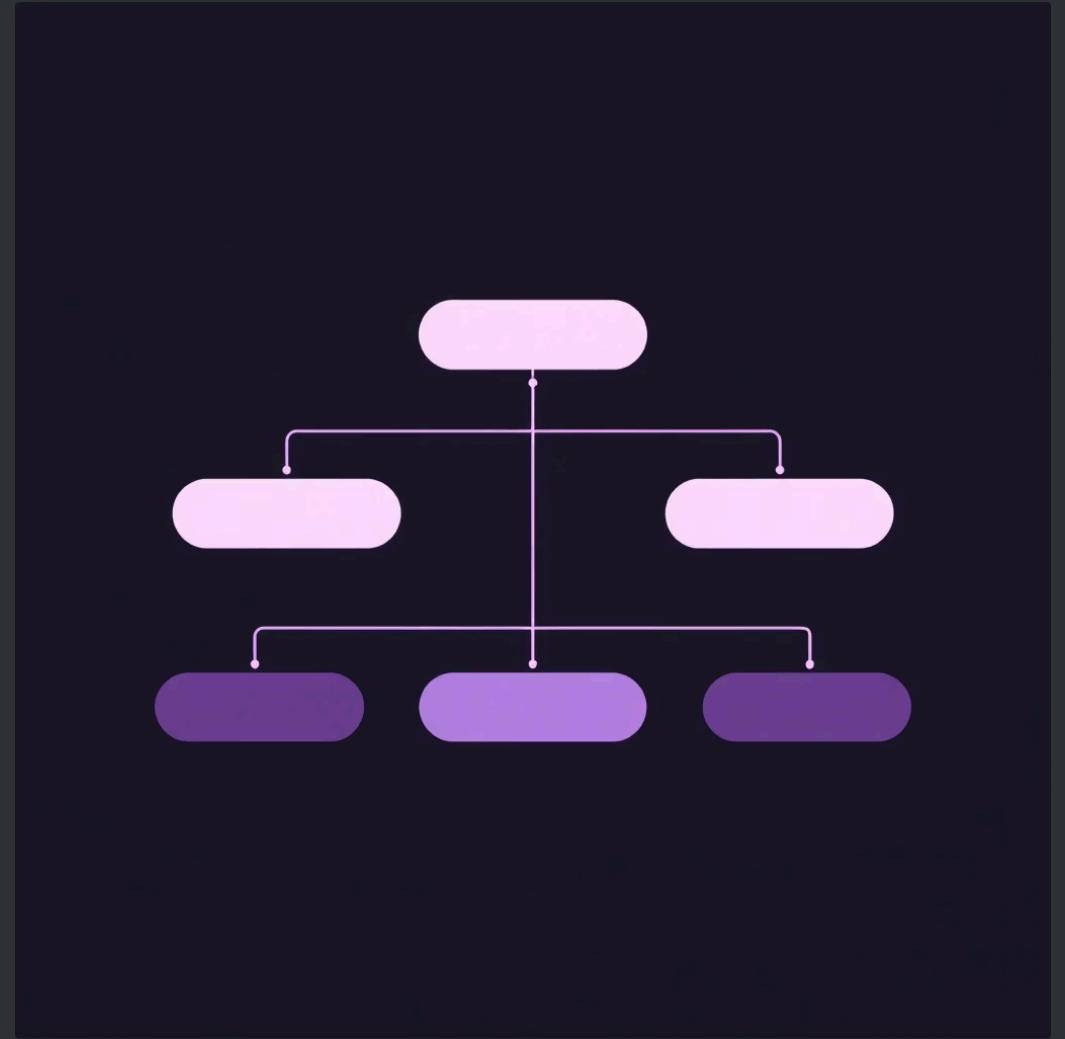
Ventajas clave:

- Alta localidad de memoria para acceso secuencial
- Búsquedas extremadamente rápidas por caché CPU
- Menor overhead de memoria por elemento

Limitaciones:

- Inserciones requieren desplazamiento de elementos
- Eliminaciones tienen costo $O(n)$ en peor caso

B-Tree (Google)



Arquitectura: Árbol balanceado multi-vía que almacena múltiples claves por nodo optimizando accesos a disco

Ventajas clave:

- Equilibrio óptimo entre búsqueda e inserción
- Escalabilidad natural con grandes volúmenes
- Operaciones de escritura eficientes

Limitaciones:

- Mayor complejidad estructural
- Saltos de memoria reducen velocidad en datasets pequeños

```

4      port;
5      ctor')
15     {
15 +   dictionary, nation, col (labl");
15     destion {
15     urities, and (clearol) }
15     rce (l:
15         <ctt:
15         cme{(
15             factale on (lpl)");
15             fop?);
15     };
15     blle);

```

Implementación Técnica en C++

01

Arquitectura sin STL

Código desarrollado en un único archivo .cpp sin dependencias de la biblioteca estándar (vector, string, map) para control total sobre el rendimiento y comportamiento de memoria

02

Estructuras Personalizadas

Implementación propia de Vector (array dinámico), FlatMapCache (con ordenamiento y búsqueda binaria), y BTreeCache (árbol balanceado con inserción y borrado lazy)

03

Generación de Trazas

Sistema de generación de patrones de acceso: aleatorio (random), secuencial (sequential) y distribución Zipf para simular comportamiento real de aplicaciones

04

Medición de Precisión

Instrumentación con `std::chrono` para capturar latencias en nanosegundos, garantizando mediciones de alta resolución temporal

```
// Ejemplo de estructura de benchmark
g++ -O2 -std=c++17 cache_compare.cpp -o cache_compare
./cache_compare [ops] [universe] [trace_type]
```

Métricas de Evaluación del Sistema

Latencia Promedio (ns/op)

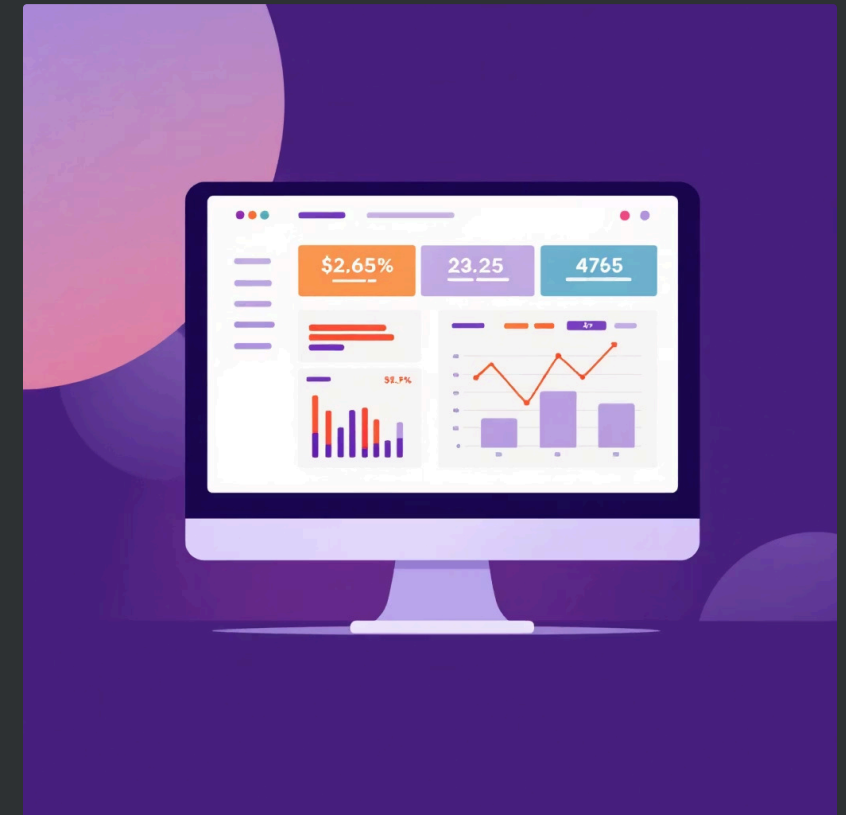
Tiempo medio que consume una operación individual de inserción o búsqueda, medido en nanosegundos. Esta métrica refleja la eficiencia bruta de cada estructura bajo carga operacional.

Tasa de Aciertos (Hit Rate)

Porcentaje de veces que el dato solicitado ya está presente en caché, evitando accesos costosos a memoria principal o disco. Un hit rate alto indica mejor efectividad de la política de caché.

Configuración Experimental

Batería de pruebas con 100,000 operaciones sobre un universo de 50,000 claves únicas, utilizando trazas aleatoria, secuencial y Zipf. Capacidad de caché configurada al 50% del universo.



- ❏ **Nota técnica:** La distribución Zipf simula patrones de acceso del mundo real donde un pequeño subconjunto de claves recibe la mayoría de las solicitudes, similar a la ley de Pareto.



Resultados Experimentales

Los experimentos revelaron diferencias significativas en el comportamiento de ambas estructuras bajo diferentes cargas de trabajo:

165.4	240.9	78.3%	77.5%
FlatMap – Latencia (ns/op)	B-Tree – Latencia (ns/op)	FlatMap Hit Rate	B-Tree Hit Rate
Tiempo promedio por operación mixta	Tiempo promedio por operación mixta	Tasa de aciertos en caché	Tasa de aciertos en caché

Operaciones de Solo Inserción

- FlatMap: 120 ns/op
- B-Tree: 210 ns/op

Operaciones de Solo Eliminación

- FlatMap: 180 ns/op
- B-Tree: 250 ns/op

Análisis Comparativo Detallado

La siguiente matriz compara el desempeño de ambas estructuras a través de múltiples dimensiones operacionales:

Operación	FlatMap	B-Tree	Ganador
Búsqueda	Rápida (menor latencia)	Media	FlatMap
Inserción	Lenta (coste por desplazamiento)	Rápida	B-Tree
Eliminación	Lenta	Estable	B-Tree
Localidad en memoria	Alta	Media	FlatMap
Escalabilidad	Media	Alta	B-Tree

Los resultados demuestran que no existe una estructura universalmente superior, sino que cada una se optimiza para patrones de acceso específicos.

Conclusiones del Estudio



Fortalezas de FlatMap

- Rendimiento superior en búsquedas con datasets pequeños a medianos
- Excelente aprovechamiento de la memoria caché del CPU
- Ideal para cargas de trabajo read-heavy con pocas modificaciones

Fortalezas de B-Tree

- Desempeño robusto en inserciones y eliminaciones masivas
- Escalabilidad probada con grandes volúmenes de datos
- Comportamiento predecible bajo cargas mixtas intensivas

- ❏ **Hallazgo clave:** Ambas estructuras logran tasas de acierto prácticamente idénticas bajo trazas equivalentes (diferencia de 0.8%), confirmando que la política de reemplazo domina sobre la estructura de datos en la efectividad de caché.

La elección óptima depende críticamente del patrón de acceso predominante y del tamaño esperado del dataset en producción.

Demostración y Ejecución Práctica

1

Compilación Optimizada

Utilizar flags de optimización del compilador para maximizar el rendimiento y habilitar características de C++17

2

Configuración de Parámetros

Especificar número de operaciones, tamaño del universo de claves y tipo de traza de acceso deseada

3

Análisis de Resultados

Revisar métricas de latencia y hit rate para validar comportamiento bajo condiciones específicas

Ejemplo de Ejecución

```
g++ -O2 -std=c++17 cache_compare.cpp -o cache_compare  
./cache_compare 100000 50000 random
```

Salida Esperada del Sistema

```
Benchmark: ops=100000 universe=50000 trace=random  
FlatMap: avg ns/op = 160.3 | hit_rate = 78%  
B-Tree : avg ns/op = 240.1 | hit_rate = 77%
```

Esta interfaz permite experimentación rápida con diferentes configuraciones para identificar el comportamiento en escenarios personalizados.

Recomendaciones y Trabajo Futuro

FlatMap para Lectura Intensiva

Seleccionar FlatMap en sistemas con alta proporción de búsquedas sobre escrituras, especialmente con datasets que caben en caché L3

B-Tree para Escritura Intensiva

Optar por B-Tree cuando el sistema requiere inserciones/eliminaciones frecuentes o maneja volúmenes que exceden la memoria caché

Direcciones de Mejora

Política LRU (Least Recently Used)

Implementar algoritmo de reemplazo LRU para mejorar la tasa de aciertos en patrones de acceso con localidad temporal

Sistema Híbrido Adaptativo

Combinar ambas estructuras en un sistema que seleccione dinámicamente la estructura óptima según el patrón de acceso detectado en tiempo real

Optimización de Prefetching

Agregar predicción de accesos futuros para pre-cargar datos en la estructura más adecuada antes de la solicitud

"La arquitectura óptima de caché no existe en aislamiento, sino que emerge de la comprensión profunda de los patrones de acceso específicos de cada aplicación."